````
---
title: "Geostatistcs"
output: html_notebook
---


```{r}
## A qualitative palette
my_colors <- c("#66C2A5", "#FC8D62", "#8DA0CB", "#E78AC3", "#A6D854",
"#FFD92F",
               "#E5C494", "#B3B3B3")
## A sequential palette
my_colors_div <- c("#FFF5EB", "#FEE6CE", "#FDD0A2", "#FDAE6B", "#FD8D3C",
                   "#F16913", "#D94801", "#A63603", "#7F2704")
```


```{r}
library(geoR)
```
```{python}
import pandas as pd

# Read the "stations.txt" file
filename = "stations.txt"

# Define new column names
column_names = ["ID", "Nom", "Latitude", "Longitude", "Altitude"]

# Read the data into a pandas DataFrame, skip the first six rows
df = pd.read_fwf(filename, skiprows=18, header=None, names=["STAID",
"STANAME", "CN", "LAT", "LON", "HGHT"], colspecs=[(0, 5), (6, 46), (47,
49), (50, 59), (60, 70), (71, 75)])

# Drop the "CN" column
df = df.drop(columns=["CN"])
# Rename columns
df.columns = column_names



# Save the DataFrame to a CSV file
df.to_csv("stations.csv", index = False)


```

```{r}
# Read the CSV file
stations <- read.csv("stations.csv")

# Display the first few rows using head()
head(stations)

```

```{python}
````
````

```
# Define a function to convert degrees, minutes, seconds to decimal degrees
def convert_dms_to_dd(dms):
    parts = [float(part) for part in dms.split(':')]
    dd = parts[0] + parts[1] / 60 + parts[2] / 3600
    return dd

# Read the CSV file
stations = pd.read_csv("stations.csv")

# Convert Latitude and Longitude to Decimal Degrees
stations['Latitude'] = stations['Latitude'].apply(convert_dms_to_dd)
stations['Longitude'] = stations['Longitude'].apply(convert_dms_to_dd)

# Display the updated data
print(stations.head())
stations.to_csv("stations.csv", index=False)
```

```{r}
library(geodata)
```

```{r}
# Read the CSV file
stations <- read.csv("stations.csv")

# Display the first few rows using head()
head(stations)
morocco <- gadm(country = "MAR", level = 0, path=tempdir())
plot(morocco)
points(stations$Longitude, stations$Latitude, pch = 15, col = my_colors[1])
```

```{r}
# Load necessary library
library(data.table)

# Function to process each file
process_file <- function(file_path) {
  # Read the file into a data.table
  dt <- fread(file_path, skip=19)

  # Extract the station ID from the file name
  station_id <- as.numeric(substr(basename(file_path), 11, 14))

  # Remove the SOUID column and add numer_sta with constant value
  dt[, `:=`(numer_sta = station_id, SOUID = NULL)]

  # Rename the TX column to t
  setnames(dt, "TX", "t")

  # Save the processed data.table to a new file
  write.csv(dt, paste0("processed_", basename(file_path)), row.names =
FALSE)
}

# List all files in the directory
```

```
files <- list.files(pattern = "TX_STAID\\d{6}\\.txt")

# Process each file
for (file in files) {
  process_file(file)
}



```


```{r}
# List all processed files in the directory
processed_files <- list.files(pattern = "processed_TX_STAID\\d{6}\\.txt")

# Read the first processed file to get the header
first_file <- fread(processed_files[1])
header <- colnames(first_file)

# Create an empty data.table to store the concatenated data
result_dt <- data.table()

# Iterate through processed files, read them, and bind rows
for (file in processed_files) {
  dt <- fread(file)
  result_dt <- rbind(result_dt, dt)
}

# Write the result data.table to a CSV file
write.csv(result_dt, "combined_processed_files.csv", row.names = FALSE)

```

```{r}
# Read the combined processed files CSV into a data frame
final_data <- read.csv("combined_processed_files.csv")

# Extract columns "numer_sta", "DATE", and "t"
data <- final_data[, c("numer_sta", "DATE", "t")]

# Remove rows where temperature is -9999
data <- data[data$t != -9999, ]

data$t <- data$t*0.1

# Display the resulting data frame
head(data)



```

```{r}
#"DATE" is in the format YYYYMMDD
data$date <- as.Date(substr(data$DATE, 1, 8), format = "%Y%m%d")

```

```{r}
head(data)
```

```{r}
# Reorder columns
data <- data[, c("numer_sta", "date", "t")]

# Remove the "DATE" column
data <- data[, !names(data) %in% c("DATE")]

# Save the DataFrame to a new CSV file
write.csv(data, "final_data.csv", row.names = FALSE)

```

```{r}
# Read the stations.csv file
stations <- read.csv("stations.csv")

# Select specific locations
selected_locations <- c("MIDELT", "OUARZAZATE", "NOUASSEUR")

# Filter the data for the selected locations
selected_data <- data[data$numer_sta %in% stations$ID[stations$Nom %in%
selected_locations], ]

# Extract unique station IDs for plotting
unique_ids <- unique(selected_data$numer_sta)

# Loop through each station ID and plot the time series
matplot(selected_data[selected_data$numer_sta %in% unique_ids, c("date",
"t")],
        type = "l", lty = 1, col = 1:length(unique_ids),
        xlab = "Date", ylab = "Temperature (°C)",
        main = "Temperature Time Series at Selected Locations",
        pch = 16, cex = 0.8)

# Add a legend
legend("topright", legend = stations$Nom[stations$ID %in% unique_ids],
       col = 1:length(unique_ids), lty = 1, cex = 0.8)


```

```{r}
# Read the final_data.csv file
final_data <- read.csv("final_data.csv")

# Convert "date" column to Date type if not already done
final_data$date <- as.Date(final_data$date)

# Specify the fixed date
fixed_date <- as.Date("1960-06-30")
```

```
# Subset the data for the specified date
onedaydata <- subset(final_data, date == fixed_date)

head(onedaydata)

```


```{r}
# "final_data" and "stations" are the DataFrames to merge
combined_merged <- merge(final_data, stations[, c("ID", "Longitude",
"Latitude", "Altitude")], by.x = "numer_sta", by.y = "ID")

# Now, combined_merged contains the columns from both DataFrames, including
"Longitude" and "Latitude"
head(combined_merged)
```


```{r}
# Create a new DataFrame
df_combined <- data.frame(combined_merged, temperature = combined_merged$t)

# Convert latitude, longitude, and altitude to (lon, lat) format
df_combined <- df_combined[, c("Longitude", "Latitude", "Altitude",
"temperature")]

# Create a geodata object
geodata_combined <- as.geodata(df_combined, coords.col = 1:2, data.col = 4,
covar.col = 3)


```
```{r}
plot(geodata_combined, qt.col = my_colors[1:4])
```

```{r}
str(geodata_combined)

```


```{r}

plot(geodata_combined$coords, geodata_combined$data, qt.col =
my_colors[1:4], pch = 16, cex = 1.5,
     xlab = "Altitude", ylab = "Temperature", main = "Temperature vs.
Altitude", covar = geodata_combined$covar$Altitude)



```

```{r}
# Scatterplot of temperature against latitude
```

```
plot(geodata_combined$coords[, "Latitude"], geodata_combined$data, pch =
16, col = my_colors[1],
     xlab = "Latitude", ylab = "Temperature", main = "Temperature vs.
Latitude")

# Fit a linear model to assess trend
model <- lm(geodata_combined$data ~ geodata_combined$coords[, "Latitude"])
abline(model, col = "red")

# Print summary of the linear model
summary(model)

```
```{r}


# Compute empirical variogram using variog
variogram_result <- variog(coords = geodata_combined$coords, data =
geodata_combined$data)

# Plot the empirical variogram
plot(variogram_result, main = "Empirical Variogram", pch = 16, col =
my_colors[1])

```